



KỸ THUẬT LẬP TRÌNH

Chương 3: Hàm và tổ chức chương trình

Nội dung

1. Tổ chức chương trình thành các hàm
2. Tham số kiểu con trỏ
3. Độ quy
4. Bài tập thực hành

3.1. Tổ chức chương trình thành các hàm

Khái niệm về hàm

- Một hàm trong C được hiểu theo nghĩa là một “*Routine*” hoặc “*subprogram*”
- Hàm là một đơn vị độc lập trong C
 - Không được xây dựng hàm bên trong 1 hàm khác
 - Mỗi hàm có thể có các biến, hằng, mảng riêng
- Một chương trình viết bằng C gồm 1 hoặc nhiều hàm, trong đó có 1 hàm chính là hàm “*main()*”
- Hàm có thể có giá trị trả về (kết quả của hàm) hoặc không có giá trị trả về (chỉ đơn thuần thực hiện 1 công việc nào đó)
- Hàm có thể có hoặc không có tham số

Khai báo hàm

Nguyên mẫu hàm (prototype của hàm)

- **Prototype** hàm chỉ rõ các đặc điểm chính
 - + Tên của hàm
 - + Số lượng và kiểu của từng tham số hàm sẽ nhận + Giá trị trả về sau khi hàm kết thúc.
 - + Phải khai báo prototype của hàm trước khi sử dụng hàm -> thường khai báo nguyên mẫu ở đầu chương trình.
- **Prototype** hàm **không** cho thấy hàm sẽ làm những gì
- Công thức khai báo:

Kiểu_hàm Tên_hàm (**Kiểu_tham_số_1**, **Kiểu_tham_số_2**, ...);

Cài đặt hàm

- Xác định chính xác những lệnh mà hàm phải thực hiện.
- Thường được cài đặt ở cuối chương trình hoặc đặt trong 1 file thư viện riêng
- Cách cài đặt:

```
Kiểu_hàm Tên_hàm (Kiểu_1 Tên_tham_số_1,  
                  Kiểu_2 Tên_tham_số_2,...)  
{  
    - Khai báo biến, hằng cục bộ trong hàm  
    - Các lệnh hàm sẽ thực hiện  
  
    return <kết quả của hàm>;  
}
```

Ví dụ 1

```
//in ra cac so nguyen to <=N
#include <stdio.h>
#include <conio.h>
int i, N;
int nguyento (int) ; //prototype cua ham
void main ()
do
{
    printf("Nhap so nguyen N: "); //sopt
    scanf("%d", &N);
} while(N<=0);
for(i=2; i<=N; i++){
    if( nguyento(i) != 0)
        printf("%8d", i);
}
getch();
}
```

```
//cai dat ham
int nguyento(int x)
{
    int j = 2;
    while (j<=x/2 && x%j!=0) j++;
    return (j>x/2)? 1:0;
}
```

Hàm **nguyento()**
Được thực hiện
bao nhiêu lần?

Quy tắc hoạt động của hàm

- Lời gọi hàm có dạng tổng quát như sau:
Tên_hàm ([danh sách tham số thực])
- Số lượng tham số thực trong lời gọi hàm phải bằng số lượng tham số hình thức (trong khai báo hàm)
- Kiểu của các tham số thực phải tương ứng với kiểu của tham số hình thức
- Khi gặp 1 lời gọi hàm tại 1 vị trí nào đó trong chương trình, máy sẽ dời vị trí đó chuyển đến thực hiện các lệnh của hàm được gọi

Quy tắc hoạt động của hàm (tt)

Thứ tự thực hiện khi có 1 lời gọi hàm

- Cấp phát bộ nhớ cho các biến cục bộ
 - Gán giá trị của tham số thực sự cho tham số hình thức
 - Thực hiện các lệnh trong thân của hàm
 - Gặp lệnh **return** hoặc dấu **}** kết thúc hàm thì xóa vùng nhớ đã cấp cho các biến cục bộ và rời khỏi hàm -> trở về vị trí đã dừng sau lời gọi hàm.
- Nếu thoát khỏi hàm từ câu lệnh **return** có chứa biểu thức thì giá trị của biểu thức được gán cho hàm. Giá trị của hàm sẽ được sử dụng trong các biểu thức chứa nó.

Tham số hình thức và tham số thực

- **Tham số hình thức:** Là tên của tham số được sử dụng khi khai báo hoặc cài đặt hàm
- **Tham số thực sự:** Là tên và giá trị của tham số được truyền cho hàm trong lời gọi hàm

```
int dayso[100], sopt;  
void NhapMang(int n, int a[])  
{  
    int i;  
    for(i=0; i<n; i++)  
    {  
        printf("Nhap phan tu a[%d]", &a[i]);  
        scanf("%d", &a[i]);  
    }  
}  
int main()  
{  
    printf("Nhap so phan tu mang:");  
    scanf("%d", &sopt);  
    NhapMang(sopt, dayso);  
    ....  
    ....  
    return 1;  
}
```

Tham số hình
thức

Tham số thực sự

Một số lưu ý

- Khi hàm không khai báo rõ kiểu thì nó mặc định hiểu là hàm có kiểu `int`
- Không nhất thiết phải khai báo `prototype` của hàm (nếu cài đặt hàm trước khi có lời gọi hàm)
- `Prototype` của hàm thực chất là dòng đầu tiên của phần cài đặt hàm nhưng có thêm dấu `;` ở cuối
- Trong khai báo `prototype` của hàm có thể bỏ đi tên của các tham số hình thức
- Trường hợp xây dựng hàm không trả về giá trị gì thì nên khai báo rõ kiểu của hàm có là kiểu `void`

Bài tập

1. Viết chương trình dạng hàm tìm GTLN của 2 số a và b (a, b được nhập vào từ bàn phím)
2. Viết hàm giải phương trình bậc nhất: $ax+b=0$
3. Viết chương trình dạng hàm giải phương trình bậc hai 1 ẩn: $ax^2 + bx + c = 0$
4. Viết chương trình dạng hàm tính TBC của 1 dãy số $a_1, a_2, \dots, a_n$ (N nhập vào từ bàn phím)
5. Viết chương trình dạng hàm tìm UCLN, BCNN của 2 số nguyên dương a, b

3.2. Truyền tham số cho hàm

- Tham số thực sự và tham số hình thức (nhắc lại)
- Có 2 cách truyền tham số cho hàm
 - Truyền theo tham trị (mặc định): Giá trị của tham số thực sự không bị thay đổi sau khi hàm kết thúc.
 - Truyền theo tham chiếu: Giá trị của tham số có thể bị thay đổi sau khi hàm kết thúc.

Truyền tham số theo tham trị

```
#include <stdio.h>
#include <conio.h>
void doicho (int a, int b)
{
    int t;
    t = a; a = b; b = t;
    printf("a va b trong doicho() la: %d va %d", a, b);
}

int main() {
    int a = 10, b = 2;
    doicho(a, b) ;
    printf("a va b ngoai doicho() la: %d va %d", a, b);
    return 0;
}
```

Truyền tham số theo tham chiếu

- Khi xây dựng hàm cần đặt dấu & trước tham số hình thức

```
#include <stdio.h>
#include <conio.h>
void doicho(int &a, int &b)
{
    int t;
    t = a; a = b; b = t;
    printf("a va b trong doicho() la: %d va %d", a, b);
}

int main()
{
    int a = 10, b=20;
    doicho(a, b);
    printf("a va b ngoai doicho() la: %d va %d", a, b);
    return 0;
}
```

Con trỏ

- Là một biến dùng để chứa địa chỉ
- Có nhiều loại con trỏ tương ứng với các kiểu địa chỉ khác nhau
 - Biến kiểu `int` -> sử dụng `con trỏ kiểu int`
 - Biến kiểu `float` -> sử dụng `con trỏ kiểu float`
 - Biến kiểu `char` -> sử dụng `con trỏ kiểu char`
- Cú pháp khai báo con trỏ

`kiểu_dữ_liệu *tên_con_trỏ;`

- Ví dụ

```
int i, j, *pi, *pj;  
pi = &i;    /* pi là con trỏ chứa địa chỉ biến i */  
pj = &j;    /* pj là con trỏ chứa địa chỉ biến j */
```

Con trỏ (tt)

- Giả sử có

- px là con trỏ đến biến x, thì các cách viết x và *px là tương đương nhau

- Ví dụ

```
int x, y, *px, *py;  
px = &x;  
py = &y;  
x = 3;      /*tương đương với *px = 3 */  
y = 5;      /*tương đương với *py = 5 */  
/* Các câu lệnh dưới đây là tương đương: */  
x = 10 * y;  
*px = 10 * y;  
x = 10 * (*py);  
*px = 10 * (*py);
```


Hàm có tham số là con trỏ

```
#include <stdio.h>
void hoan_vi(int a, int b); /* prototype hàm*/
void main()
{
    int n=10, p=20;
    printf(" Truoc khi goi hàm : %d %d\n", n, p);
    hoan_vi(n, p);
    printf(" Sau khi goi hàm   : %d %d\n", n, p);
}

void hoan_vi(int a, int b)
{
    int t;
    printf(" Truoc khi hoan vi : %d %d\n", a, b);
    t=a;
    a=b;
    b=t;
    printf(" Sau khi hoan vi   : %d %d\n", a, b);
}
```

Hàm có tham số là con trỏ (tt)

- Chương trình trên cho kết quả không đúng
- Tại sao ?
- Do cơ chế biến cục bộ hay tham số hình thức bị giải phóng bộ nhớ khi hàm kết thúc ?
- Truyền tham số thực cho hàm là địa chỉ biến thay vì truyền giá trị biến
 - Sử dụng tham số là con trỏ!

Hàm có tham số là con trỏ (tt)

```
#include <stdio.h>
void hoan_vi(int *a, int *b);
void main()
{   int n = 10, p = 20;
    printf(" Truoc khi goi hàm : %d %d\n", n, p);
    hoan_vi(&n, &p);
    printf(" Sau khi goi hàm   : %d %d\n", n, p);
}

void hoan_vi(int *a, int *b) // a và b bây giờ là 2 địa chỉ
{   int t;
    printf(" Truoc khi hoán vị : %d %d\n", *a, *b);
    t = *a; /* t nhận giá trị chưa trong địa chỉ a */
    *a = *b;
    *b = t;
    printf(" Sau khi hoán vị   : %d %d\n", *a, *b);
}
```

Hàm có tham số là con trỏ (tt)

Khi nào thì dùng tham số là con trỏ ?

- Cần phân biệt hai loại tham số hình thức
 - Tham số hình thức chỉ nhận giá trị truyền vào để hàm thao tác, trường hợp có thể gọi là tham số vào.
 - Tham số hình thức dùng để chứa kết quả của hàm, trường hợp này có thể gọi là tham số ra
- Đối với tham số ra ta phải sử dụng kiểu con trỏ.
- Thường dùng trong trường hợp:
 - Sử dụng hàm để thay đổi giá trị của một biến
 - Truyền một mảng vào cho hàm
 - Hàm trả về nhiều kết quả

3.3. Hàm đệ quy

- Ngôn ngữ C cho phép 1 hàm gọi tới chính nó từ một điểm nào đó trong thân của hàm.
- Những hàm có lời gọi hàm tới chính nó được gọi là hàm đệ quy.

```
int giaithua(int n)
{
    if(n== 0 || n== 1)
        return 1;
    else
        return n * giaithua(n-1);
}

void main()
{
    int n;
    printf ("Nhap N = "); scanf("%d", &n);
    printf("%d! = %ld", n, giaithua(n) );
    getch();
}
```

3.3. Hàm đệ quy (tt)

Điều gì xảy ra nếu có lời gọi hàm sau

$k = \text{giaithua}(-1);$

Khắc phục ?

- Hạn chế của hàm đệ quy
 - Dùng nhiều bộ nhớ
- Hãy viết lại hàm *giai_thua* sử dụng vòng lặp
- So sánh hai cách viết đệ quy và lặp

3.3. Hàm đệ quy (tt)

- Hàm đệ quy thường phù hợp để giải quyết các bài toán có đặc trưng
 - Bài toán dễ dàng giải quyết trong một số trường hợp riêng, đó chính là **điều kiện dừng đệ quy**
 - Trong trường hợp tổng quát, bài toán suy về cùng dạng nhưng giá trị tham số bị thay đổi
- **Ví dụ:** tìm USCLN của hai số nguyên dương
 - nếu $x = y$ thì $usc(x, y) = x$
 - nếu $x > y$ thì $usc(x, y) = usc(x-y, y)$
 - nếu $x < y$ thì $usc(x, y) = usc(x, y-x)$

Cách xây dựng hàm đệ quy

- Thường được xây dựng theo thuật toán sau:

```
if(truong_hop_suy_bien)
{
    //trinh bay thuat toan khi suy bien
}
else
{
    //goi toi ham de quy (dang viet) voi gia tri khac cua tham so
}
```

Ví dụ:

```
int usc(int x, int y)
{
    if (x == y)
        return (x);
    else if (x > y)
        return usc(x-y, y);
    else
        return usc(x, y-x);
}
```


3.4. Bài tập hàm đệ quy

- Hãy viết chương trình sử dụng hàm đệ quy để tạo dãy số Fibonacci:
 - Dãy số Fibonacci là dãy số $F_1, F_2, F_3, \dots, F_n$ được tạo ra với công thức:
 - $F_n = F_{n-1} + F_{n-2}$ Với $F_1 = 1, F_2 = 1$
- Ví dụ: 1, 1, 2, 3, 5, 8, 13, 21, ...

```
int Fibonacci(int n) {  
    if (n <= 1)  
        return 1;  
    else  
        return Fibonacci(n-1) + Fibonacci(n-2);  
}
```

Bài tập

1. Viết hàm đệ quy tính tổng từ 1 đến N ($N \geq 0$)
2. Viết hàm đệ quy tính $N!$
3. Viết hàm đệ quy tìm UCLN, BCNN của 2 số nguyên a,b